

MAT31-6

Math for Data Practice

Regression, regularization, and model evaluation — from scratch

Fitting a model is solving a geometry problem; trusting it is a probability problem.

Albert School — 18 hours

Preface

You arrive at this course already holding every tool it needs. From **Linear Algebra II** (MAT21-1) you know eigenvalues, eigenvectors, and the diagonalization $A = PDP^{-1}$ that exposes the directions a matrix merely scales. From **Linear Algebra III** (MAT31-1) you know the scalar product and the geometry it unlocks: orthogonality, the **spectral theorem** $A = PDP^T$ for symmetric matrices, and — the keystone for us — **orthogonal projection** and the **best-approximation theorem**, which says the projection of a vector onto a subspace is the closest point of that subspace. From **Probability & Statistics II** (MAT31-2) you know random variables, expectation and variance, the idea of an **estimator**, and the distinction between a quantity computed on a sample and the population quantity it estimates.

This course assembles those three streams into the single most-used procedure in all of data science: **fitting a linear model to data, and then deciding whether to believe it**. The whole arc has two halves, and the tagline names them. The **fitting** half is pure linear algebra: we write the model $y = X\beta + \varepsilon$, minimize the squared error, and discover — with no new machinery — that the answer is exactly the orthogonal projection of MAT31-1. The **trusting** half is probability: a model that fits the data it was trained on may be worthless on new data, and separating genuine signal from memorized noise is the **bias–variance tradeoff**, diagnosed by **cross-validation** and controlled by **regularization**.

We build everything from scratch. Where a data-science library would offer a one-line **fit**, we derive the normal equations, invert by hand on small examples, watch what multicollinearity does to the arithmetic, and see geometrically why an L_1 penalty produces zeros while an L_2 penalty does not. The reward is that nothing in the modern regression toolkit will ever be a black box to you again.

Keep two pictures in mind throughout. The first is a vector y casting a shadow onto a subspace — that shadow is the fit, and the gap is the error. The second is a U-shaped curve: error falling as a model grows more flexible, then rising again as it starts to memorize noise. Master those two pictures and the rest is detail.

Contents

Preface	2
1 Linear regression from scratch	4
1.1 The linear model	4
1.2 What “best” means: least squares	4
1.3 The normal equations	5
1.4 The pseudo-inverse solution	5
1.5 Geometric interpretation: regression is projection	6
2 Residual analysis	7
2.1 The fundamental orthogonality	7
2.2 Decomposing the variance: R^2	7
2.3 Why R^2 always rises, and the adjusted fix	8
2.4 Residual plots and heteroscedasticity	9
3 Multicollinearity	10
3.1 The geometry of near-dependence	10
3.2 Why small eigenvalues destabilize the estimate	10
3.3 Variance inflation factors	11
4 Ridge regression	11
4.1 The penalized objective	11

4.2	The constraint picture: a ball	12
4.3	The closed-form solution	12
4.4	Effect of λ on eigenvalues and shrinkage	13
5	Lasso regression	13
5.1	The L1 penalty	14
5.2	Why the diamond produces sparsity	14
6	Comparing L1 and L2 regularization	15
7	Cross-validation	16
7.1	Why training error underestimates test error	16
7.2	k-fold cross-validation	16
7.3	Selecting λ by cross-validation	17
8	The bias–variance tradeoff	18
8.1	The decomposition	18
8.2	The tradeoff and its U-shaped curve	18
8.3	The numerical demonstration	19
9	The full regression pipeline	20

1 Linear regression from scratch

Suppose a property company records, for each of several flats, its floor area and its monthly rent. Plotting rent against area, the points fall roughly along a line, and we would like the **best** line — one that lets us predict the rent of a flat we have not seen. “Best” must be made precise, and once it is, the answer will turn out to be a projection you already know how to compute. We begin with this concrete goal and abstract it only as far as needed.

1.1 The linear model

We observe n data points. Each carries a **response** (the rent) and p **features** (area, number of rooms, ...; one of them is a constant 1 that allows the line to miss the origin). Stack the responses into a vector $y \in \mathbb{R}^n$ and the features into a **design matrix** $X \in M_{n,p}(\mathbb{R})$, one row per observation:

$$X = \begin{pmatrix} 1 & x_{11} & \dots & x_{1,p-1} \\ 1 & x_{21} & \dots & x_{2,p-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \dots & x_{n,p-1} \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

The first column of ones is the **intercept** column. The **linear model** asserts that the response is a linear combination of the features, plus an unavoidable random error:

Definition 1 (Linear model)

$$y = X\beta + \varepsilon,$$

where $\beta \in \mathbb{R}^p$ is the unknown **coefficient** vector to be estimated and $\varepsilon \in \mathbb{R}^n$ is a random **error** (or **noise**) vector, assumed to have mean zero and to be uncorrelated across observations with common variance σ^2 .

Worked example — rent against area. With a single feature (area a_i) and an intercept, row i of X is $(1, a_i)$ and $\beta = (\beta_0, \beta_1)$. The model reads $y_i = \beta_0 + \beta_1 a_i + \varepsilon_i$: a base rent β_0 , plus β_1 euros per square metre, plus noise. Fitting the model means choosing the intercept β_0 and slope β_1 so the line $\beta_0 + \beta_1 a$ tracks the cloud of points as closely as possible.

Common pitfall The model is “linear” in the **coefficients** β , not necessarily in the raw measurements. Adding a column a_i^2 to X fits a parabola $\beta_0 + \beta_1 a + \beta_2 a^2$ — still a linear model, because it is linear in $(\beta_0, \beta_1, \beta_2)$. Everything in this course applies the moment you can write your predictor as $X\beta$, however exotic the columns of X .

1.2 What “best” means: least squares

For a candidate β , the prediction for observation i is the i -th entry of $X\beta$, and the **residual** — how far the prediction misses — is $y_i - (X\beta)_i$. We want all residuals small at once. Squaring them (so that positive and negative misses cannot cancel, and large misses are penalized heavily) and summing gives a single number to minimize.

Definition 2 (Ordinary least squares) The **ordinary least squares** (OLS) estimate is the coefficient vector minimizing the sum of squared residuals:

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^p} \|y - X\beta\|^2 = \arg \min_{\beta} \sum_{i=1}^n (y_i - (X\beta)_i)^2.$$

The notation $\|\cdot\|$ is the ordinary Euclidean norm of MAT31-1: the quantity we minimize is the **squared distance** between the observed vector y and the predicted vector $X\beta$. This is the first hint that the problem is geometric.

1.3 The normal equations

Minimizing a smooth function means setting its gradient to zero. Expanding the objective and differentiating gives a linear system — no calculus of variations, just one gradient.

Theorem 3 (Normal equations) Every minimizer $\hat{\beta}$ of $\|y - X\beta\|^2$ satisfies

$$X^T X \hat{\beta} = X^T y.$$

Proof. Write $f(\beta) = \|y - X\beta\|^2 = (y - X\beta)^T (y - X\beta)$. Expanding,

$$f(\beta) = y^T y - 2\beta^T X^T y + \beta^T (X^T X) \beta.$$

Differentiating with respect to β (using $\nabla(b^T \beta) = b$ and $\nabla(\beta^T A \beta) = 2A\beta$ for symmetric $A = X^T X$),

$$\nabla f(\beta) = -2X^T y + 2X^T X \beta.$$

Setting $\nabla f = 0$ gives $X^T X \beta = X^T y$. The Hessian is $2X^T X$, which is positive semi-definite (for any v , $v^T X^T X v = \|Xv\|^2 \geq 0$), so the critical point is indeed a global minimum. $\square$

The system $X^T X \hat{\beta} = X^T y$ is small — only p equations in p unknowns — even when n is enormous. The big matrix X ($n \times p$) collapses into the manageable $p \times p$ matrix $X^T X$. This is why regression scales.

Worked example — fitting a line by hand. Three points $(1, 1), (2, 2), (3, 2)$, model $y = \beta_0 + \beta_1 x$. Then

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad y = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}, \quad X^T X = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}, \quad X^T y = \begin{pmatrix} 5 \\ 11 \end{pmatrix}.$$

Solving $\begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \hat{\beta} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$ gives $\hat{\beta} = (2/3, 1/2)$, i.e. the best-fit line is $\hat{y} = \frac{2}{3} + \frac{1}{2}x$. Check: at $x = 1, 2, 3$ it predicts $\frac{7}{6}, \frac{5}{3}, \frac{13}{6}$, with residuals summing to zero — as the next chapter proves they must.

1.4 The pseudo-inverse solution

When can we simply invert? Exactly when the features carry no redundancy.

Proposition 4 (Pseudo-inverse solution) If the columns of X are linearly independent — X has **full column rank** p — then $X^T X$ is invertible and the least-squares solution is unique:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

The matrix $X^+ = (X^T X)^{-1} X^T$ is the **Moore–Penrose pseudo-inverse** of X : a stand-in for an inverse when X is rectangular and cannot have one.

Proof. If $X^T X v = 0$ then $\|Xv\|^2 = v^T X^T X v = 0$, so $Xv = 0$; full column rank forces $v = 0$. Hence $X^T X$ has trivial kernel and, being square, is invertible. Multiplying the normal equations by $(X^T X)^{-1}$ isolates $\hat{\beta}$. $\square$

Common pitfall X itself is $n \times p$ and almost never square, so “ X^{-1} ” is meaningless — writing $\hat{\beta} = X^{-1}y$ is a classic blunder. The pseudo-inverse $X^+ = (X^T X)^{-1} X^T$ is the correct object: it left-inverts X ($X^+ X = I_p$) without X being invertible. When the columns are **dependent**, $X^T X$ is singular and no unique solution exists — the situation that motivates regularization in Chapter 4.

1.5 Geometric interpretation: regression is projection

Here the threads from MAT31-1 tie together. As β ranges over $\mathbb{R}^p$, the vector $X\beta$ ranges over all linear combinations of the columns of X — that is, over the **column space** $\text{Col}(X)$, a subspace of $\mathbb{R}^n$ of dimension p (when X has full rank). Minimizing $\|y - X\beta\|$ is therefore asking: **which point of the subspace $\text{Col}(X)$ is closest to y ?**

By the best-approximation theorem (MAT31-1), the answer is the **orthogonal projection** of y onto $\text{Col}(X)$.

Theorem 5 (Regression as projection) The fitted vector $\hat{y} = X\hat{\beta}$ is the orthogonal projection of y onto $\text{Col}(X)$:

$$\hat{y} = Hy, \quad \text{where} \quad H = X(X^T X)^{-1} X^T.$$

The **hat matrix** H (it “puts the hat on y ”) is the projection matrix onto $\text{Col}(X)$; it is symmetric ($H^T = H$) and idempotent ($H^2 = H$).

Proof. Substitute $\hat{\beta} = (X^T X)^{-1} X^T y$ into $\hat{y} = X\hat{\beta}$ to get $\hat{y} = X(X^T X)^{-1} X^T y = Hy$. This is exactly the projection-matrix formula $M(M^T M)^{-1} M^T$ of MAT31-1 with $M = X$. Symmetry and idempotence are the defining marks of an orthogonal projection. $\square$

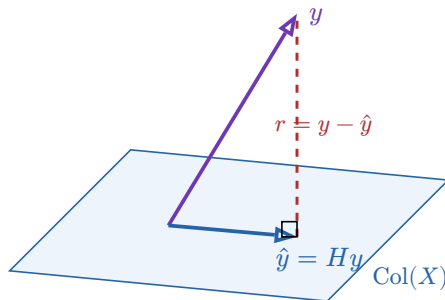


Figure 1: Linear regression is orthogonal projection. The response y rarely lies in the column space $\text{Col}(X)$ of achievable predictions. The fit $\hat{y} = Hy$ is its shadow on that subspace — the closest reachable point — and the residual $r = y - \hat{y}$ is perpendicular to the subspace (right angle marked). Minimizing squared error and dropping a perpendicular are the same act.

This single picture is the conceptual centre of the whole course: the fit is a shadow, the residual is the perpendicular gap, and “least squares” is just “shortest perpendicular”. Every later idea — R^2 , multicollinearity, regularization — is a comment on this picture.

Exercises

1. Fit $y = \beta_0 + \beta_1 x$ to the points $(0, 1), (1, 1), (2, 4)$ by forming and solving the normal equations.
2. Show that for any full-rank X , the hat matrix satisfies $HX = X$. Interpret this: projecting a vector already in $\text{Col}(X)$ leaves it unchanged.
3. Verify directly that $H = X(X^T X)^{-1} X^T$ is idempotent, $H^2 = H$.
4. Why does adding a column of ones to X correspond to allowing an intercept? What model do you get with **no** intercept column?

2 Residual analysis

Fitting produces numbers; **residual analysis** asks whether to trust them. The residual vector records what the model failed to explain, and — because the fit is a projection — it has a rigid geometric structure we can exploit both to summarize fit quality (R^2) and to detect when the model is wrong.

2.1 The fundamental orthogonality

Definition 6 The **residual vector** is $r = y - \hat{y} = y - X\hat{\beta}$.

Proposition 7 (Residuals are orthogonal to the columns) The residual vector is orthogonal to every column of X :

$$X^T r = 0.$$

Equivalently, $r \perp \text{Col}(X)$. If X contains an intercept column of ones, then in particular $\sum_i r_i = 0$: the residuals sum to zero.

Proof. From the normal equations, $X^T r = X^T (y - X\hat{\beta}) = X^T y - X^T X \hat{\beta} = 0$. The column of ones $\mathbf{1}$ is one of the columns of X , so $\mathbf{1}^T r = \sum_i r_i = 0$. $\square$

This is not a computational accident; it **is** the projection picture restated. The residual is the perpendicular dropped from y to $\text{Col}(X)$, so by construction it is orthogonal to everything in that subspace. Every diagnostic below leans on this fact.

Common pitfall Orthogonality $X^T r = 0$ is **guaranteed by the arithmetic** — it holds for any data whatsoever, even data with no linear relationship at all. So finding $\sum_i r_i = 0$ confirms only that you solved the normal equations correctly; it is **not** evidence that the model is good. The genuine diagnostics are R^2 and the **shape** of the residuals, below.

2.2 Decomposing the variance: R^2

How much of the variation in y did the model capture? Measure variation by sums of squared deviations.

Definition 8 (Sums of squares) With $|y| = \frac{1}{n} \sum_i y_i$ the mean response,

$$\text{TSS} = \sum_{i=1}^n (y_i - |y|)^2, \quad \text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{ESS} = \sum_{i=1}^n (\hat{y}_i - |y|)^2,$$

the **total**, **residual**, and **explained** sums of squares.

Proposition 9 (ANOVA decomposition) If the model contains an intercept, then

$$\text{TSS} = \text{ESS} + \text{RSS}.$$

Proof. Write $y - |y|\mathbf{1} = (\hat{y} - |y|\mathbf{1}) + (y - \hat{y}) = (\hat{y} - |y|\mathbf{1}) + r$. The first bracket lies in $\text{Col}(X)$ (both $\hat{y}$ and the constant vector $|y|\mathbf{1}$ do, since the intercept column is present), while $r \perp \text{Col}(X)$. The two are orthogonal, so Pythagoras (MAT31-1) gives $\|y - |y|\mathbf{1}\|^2 = \|\hat{y} - |y|\mathbf{1}\|^2 + \|r\|^2$, which is $\text{TSS} = \text{ESS} + \text{RSS}$. $\square$

Definition 10 (Coefficient of determination)

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}}.$$

It is the fraction of the variance of y explained by the model. With an intercept, $R^2 \in [0, 1]$: $R^2 = 1$ means a perfect fit ($r = 0$), $R^2 = 0$ means the model does no better than predicting the mean $|y|$.

Worked example — reading an R^2 . For the line of the worked example in Chapter 1, with data $(1, 1), (2, 2), (3, 2)$: $|y| = \frac{5}{3}$, fitted values $\frac{7}{6}, \frac{5}{3}, \frac{13}{6}$. Then $\text{RSS} = \left(1 - \frac{7}{6}\right)^2 + \left(2 - \frac{5}{3}\right)^2 + \left(2 - \frac{13}{6}\right)^2 = \frac{1}{36} + \frac{1}{9} + \frac{1}{36} = \frac{1}{6}$, and $\text{TSS} = \left(1 - \frac{5}{3}\right)^2 + \left(2 - \frac{5}{3}\right)^2 + \left(2 - \frac{5}{3}\right)^2 = \frac{4}{9} + \frac{1}{9} + \frac{1}{9} = \frac{2}{3}$. So $R^2 = 1 - \frac{1/6}{2/3} = 1 - \frac{1}{4} = 0.75$: the line explains 75% of the variation in rent.

2.3 Why R^2 always rises, and the adjusted fix

Common pitfall R^2 **never decreases** when you add a feature — even a column of pure random noise. Geometrically, enlarging $\text{Col}(X)$ can only bring the projection closer to y , so RSS can only shrink. A high R^2 is therefore not proof of a good model: with $p = n$ columns you can drive R^2 to 1 and explain nothing. This is the single most over-interpreted number in applied statistics.

To penalize meaningless features, divide each sum of squares by its degrees of freedom before comparing.

Definition 11 (Adjusted R^2)

$$R_{\text{adj}}^2 = 1 - \frac{\text{RSS} / (n - p)}{\text{TSS} / (n - 1)}.$$

Adding a feature lowers RSS but also lowers the divisor $n - p$; the adjusted R^2 rises only if the new feature reduces RSS by **more** than the one degree of freedom it costs. It can decrease, and is the honest summary for comparing models of different sizes.

2.4 Residual plots and heteroscedasticity

The numbers R^2 and R^2_{adj} summarize **how much** is left unexplained; the **shape** of the residuals reveals **what kind** of failure occurred. A **residual plot** graphs r_i against the fitted values $\hat{y}_i$ (or against a feature).

Under the model assumptions the residuals should look like featureless noise: scattered randomly about zero with **constant** spread — a condition called **homoscedasticity**. Two departures matter.

- A **curved or structured pattern** (residuals systematically positive in the middle and negative at the ends, say) signals that the linear model is **misspecified** — the true relationship bends, and a straight line cannot follow it.
- **Heteroscedasticity**: the spread of the residuals changes across the range of $\hat{y}$ — classically a **funnel** that widens as $\hat{y}$ grows. The constant-variance assumption $\text{Var}(\varepsilon_i) = \sigma^2$ fails, which (while leaving $\hat{\beta}$ unbiased) invalidates the usual standard errors and tests.

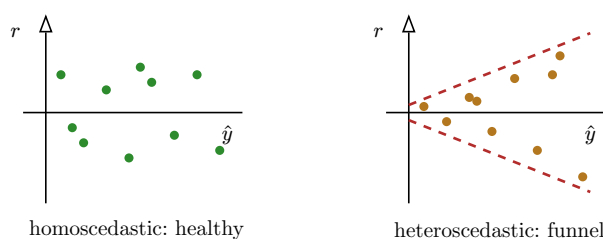


Figure 2: Two residual plots. Left: residuals scatter in a horizontal band of constant width about zero — homoscedastic, the model's assumptions hold. Right: the spread widens with $\hat{y}$ (dashed envelope) — heteroscedastic. The fit may still be unbiased, but its reported standard errors and confidence intervals (MAT31-2) can no longer be trusted.

Remark The cure for heteroscedasticity is usually to transform the response (a logarithm often stabilizes a widening funnel) or to weight observations by the inverse of their variance. The cure for a **curved** pattern is to enrich the model — add polynomial or interaction columns to X — turning the misspecification into a richer linear model.

Exercises

5. For data $(0, 1), (1, 3), (2, 3), (3, 5)$ fit a line, compute TSS, RSS, ESS, verify $\text{TSS} = \text{ESS} + \text{RSS}$, and report R^2 .
6. Show that adding any column to X cannot increase RSS. (Hint: enlarging a subspace cannot move the projection further from y .)
7. A model has $n = 100$, $p = 5$, $R^2 = 0.40$. Compute R^2_{adj} . Then recompute with $p = 50$ and the same R^2 ; comment.
8. Sketch the residual plot you would expect if the true relationship is quadratic but you fit a straight line.

3 Multicollinearity

The pseudo-inverse formula assumed the columns of X are independent. In real data they are often **nearly** dependent — two features carrying almost the same information (a flat’s area in square metres and its area in square feet, say). This **multicollinearity** does not break the arithmetic outright, but it makes the answer wildly unstable. This chapter diagnoses it.

3.1 The geometry of near-dependence

When two columns of X point in almost the same direction, the column space is “thin” in the direction they nearly share. Projecting y onto a thin, nearly degenerate subspace gives a $\hat{y}$ that is fine, but the **coordinates** $\hat{\beta}$ expressing it become huge and delicately balanced: a large positive multiple of one column almost cancelled by a large negative multiple of the other. A tiny change in y swings them violently.

This instability is visible in the spectrum of $X^T X$, the symmetric positive-semidefinite matrix at the heart of the normal equations. By the spectral theorem (MAT31-1) it has real non-negative eigenvalues; near-dependence drives the smallest of them toward zero.

Definition 12 (Condition number) For $X^T X$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$, the **condition number** is

$$\kappa(X^T X) = \frac{\lambda_1}{\lambda_p}.$$

A condition number near 1 means a well-balanced design; a very large one (thousands or more) signals severe multicollinearity.

3.2 Why small eigenvalues destabilize the estimate

In the orthonormal eigenbasis of $X^T X$ (which exists by the spectral theorem), $(X^T X)^{-1}$ has eigenvalues $1/\lambda_i$. The least-squares solution $\hat{\beta} = (X^T X)^{-1} X^T y$ therefore amplifies any component of $X^T y$ lying along the eigenvector of λ_p by the enormous factor $1/\lambda_p$.

Proposition 13 (Variance blow-up) Under the model with $\text{Var}(\varepsilon) = \sigma^2 I$, the covariance of the OLS estimate is

$$\text{Cov}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}.$$

Its eigenvalues are σ^2/λ_i . The variance of $\hat{\beta}$ in the direction of the smallest eigenvalue λ_p is σ^2/λ_p , which explodes as $\lambda_p \rightarrow 0$.

Proof. $\hat{\beta} = (X^T X)^{-1} X^T y$ is linear in y , and $\text{Cov}(y) = \sigma^2 I$. With $A = (X^T X)^{-1} X^T$, $\text{Cov}(\hat{\beta}) = A(\sigma^2 I)A^T = \sigma^2 (X^T X)^{-1} X^T X (X^T X)^{-1} = \sigma^2 (X^T X)^{-1}$, using symmetry of $(X^T X)^{-1}$. $\square$

So multicollinearity does not bias the coefficients — on average they are still right — but it inflates their **variance**: across repeated samples they swing enormously, can take absurd magnitudes, and frequently flip sign. They become uninterpretable.

Worked example — two near-identical features. Let the design (after centring) have columns c_1 and $c_2 = c_1 + \delta$ for a tiny perturbation δ . Then $X^T X = \begin{pmatrix} a & a \\ a & a+e \end{pmatrix}$ with e small, whose smaller eigenvalue is $\approx \frac{e}{2} \rightarrow 0$. The inverse has entries of order $1/e$, so $\hat{\beta}$ has entries of

order $1/e$ — gigantic and oppositely signed. Predictions $\hat{y} = X\hat{\beta}$ remain stable because the giant coefficients cancel; only the **coefficients** are meaningless.

3.3 Variance inflation factors

The condition number is a single global summary. To pin the blame on individual features, regress each feature on the others.

Definition 14 (Variance inflation factor) For feature j , let R_j^2 be the coefficient of determination of the regression of column j on all the other feature columns. The **variance inflation factor** is

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

If feature j is uncorrelated with the rest, $R_j^2 = 0$ and $\text{VIF}_j = 1$. If it is nearly a linear combination of the others, $R_j^2 \rightarrow 1$ and $\text{VIF}_j \rightarrow \infty$. The name is literal: VIF_j is the factor by which collinearity multiplies the variance of $\hat{\beta}_j$ compared with an uncorrelated design. A common rule of thumb flags $\text{VIF}_j > 10$ (i.e. $R_j^2 > 0.9$) as troublesome.

Common pitfall Multicollinearity is a property of the **features**, not of the response y — you can diagnose it from X alone, before ever looking at y . And it does not hurt **prediction**: if all you want is $\hat{y}$, collinear features are harmless. It wrecks only **interpretation** of individual coefficients. Decide which goal you have before deciding whether to worry.

Exercises

9. Compute the eigenvalues and condition number of $X^T X = \begin{pmatrix} 4 & 3.9 \\ 3.9 & 4 \end{pmatrix}$. Is this design well- or ill-conditioned?
10. Show that $\text{VIF}_j = 1$ exactly when feature j is orthogonal to the span of the other features.
11. Explain, using $\text{Cov}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}$, why centring and scaling the columns of X can improve conditioning.
12. Two features have correlation 0.99. Estimate R_j^2 and the resulting VIF.

4 Ridge regression

Multicollinearity traced the trouble to small eigenvalues of $X^T X$. **Ridge regression** attacks the disease at its root: it adds a constant to every eigenvalue, lifting them all away from zero. The price is a small, controlled bias; the reward is dramatically reduced variance — the bias-variance bargain at the heart of regularization.

4.1 The penalized objective

Definition 15 (Ridge regression) For a **regularization parameter** $\lambda \geq 0$,

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta \in \mathbb{R}^p} (\|y - X\beta\|^2 + \lambda \|\beta\|^2).$$

The added term $\lambda \|\beta\|^2 = \lambda \sum_j \beta_j^2$ is an L_2 **penalty**: it charges the fit for large coefficients, discouraging the wild values that multicollinearity produces.

At $\lambda = 0$ we recover OLS. As λ grows, the penalty dominates and the coefficients are pulled toward zero. The parameter λ tunes the trade between fitting the data and keeping coefficients small.

4.2 The constraint picture: a ball

There is an equivalent **constrained** form that makes the geometry vivid: minimizing $\|y - X\beta\|^2 + \lambda\|\beta\|^2$ is the same as

$$\min_{\beta} \|y - X\beta\|^2 \quad \text{subject to} \quad \|\beta\|^2 \leq t$$

for some budget $t = t(\lambda)$ (large $\lambda \leftrightarrow$ small t). The unconstrained objective $\|y - X\beta\|^2$ has elliptical contours centred at the OLS solution $\hat{\beta}_{\text{OLS}}$. The constraint region $\|\beta\|^2 \leq t$ is a **ball** (a disk in two dimensions) centred at the origin. The ridge solution is where the growing ellipses **first touch** the ball.

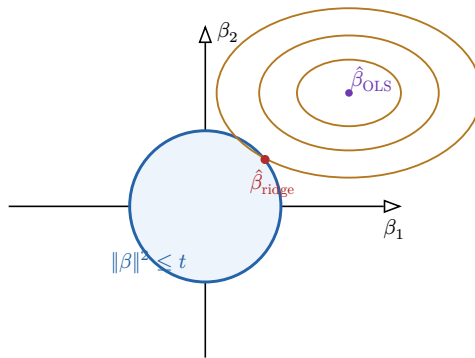


Figure 3: Ridge as a ball constraint. The orange ellipses are level sets of the least-squares error, centred at the OLS estimate $\hat{\beta}_{\text{OLS}}$. The blue disk is the budget $\|\beta\|^2 \leq t$. The ridge estimate is where an expanding ellipse first meets the disk — pulled toward the origin from $\hat{\beta}_{\text{OLS}}$. Because the disk is **round**, the contact generically lands off the axes: both coordinates shrink but neither becomes exactly zero.

4.3 The closed-form solution

The smooth L_2 penalty keeps the objective quadratic, so one gradient still solves it.

Proposition 16 (Ridge solution) For every $\lambda > 0$,

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

The matrix $X^T X + \lambda I$ is **invertible for every $\lambda > 0$** , even when $X^T X$ is singular.

Proof. The objective $g(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2$ has gradient $\nabla g = -2X^T y + 2X^T X\beta + 2\lambda\beta$. Setting it to zero, $(X^T X + \lambda I)\beta = X^T y$. For invertibility, note $X^T X$ has eigenvalues $\lambda_i \geq 0$, so $X^T X + \lambda I$ has eigenvalues $\lambda_i + \lambda \geq \lambda > 0$ — all strictly positive, hence the matrix is invertible. $\square$

That last line is the whole point: **adding λI guarantees invertibility**. Ridge regression always has a unique solution, even with perfectly collinear or more-features-than-observations ($p > n$) data, where OLS simply fails.

4.4 Effect of λ on eigenvalues and shrinkage

To see exactly **how** ridge tames the estimate, diagonalize. By the spectral theorem write $X^T X = P D P^T$ with $D = \text{diag}(\lambda_1, \dots, \lambda_p)$ and P orthogonal. Then $X^T X + \lambda I = P(D + \lambda I)P^T$, and in the eigenbasis the component of the solution along eigenvector i is scaled by

$$\underbrace{\frac{\lambda_i}{\lambda_i + \lambda}}_{\text{shrinkage factor}} \in (0, 1).$$

Proposition 17 (Directional shrinkage) Relative to OLS, ridge multiplies the coefficient along the i -th eigenvector of $X^T X$ by $\lambda_i/(\lambda_i + \lambda)$. Well-determined directions (large λ_i) are barely touched; poorly-determined directions (small λ_i — exactly the multicollinear ones) are shrunk hardest.

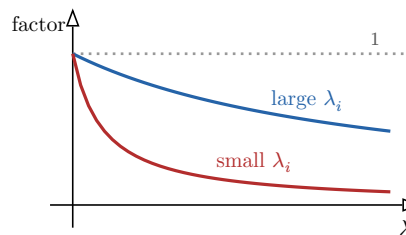


Figure 4: The shrinkage factor $\lambda_i/(\lambda_i + \lambda)$ as λ grows. For a well-determined direction (large λ_i , blue) it stays near 1: that coefficient is barely shrunk. For a poorly-determined, multicollinear direction (small λ_i , red) it collapses quickly: ridge suppresses exactly the unstable directions, which is why it cures variance blow-up.

As $\lambda \rightarrow 0$ every factor $\rightarrow 1$ and we recover OLS; as $\lambda \rightarrow \infty$ every factor $\rightarrow 0$ and $\hat{\beta}_{\text{ridge}} \rightarrow 0$.

Common pitfall Always **standardize** the features (centre and scale to unit variance) before applying ridge. The penalty $\lambda \sum_j \beta_j^2$ treats all coefficients alike, but a coefficient's size depends on its feature's units: measuring area in square feet rather than square metres rescales β_j and changes how hard the penalty hits it. The intercept is conventionally left **unpenalized** — there is no reason to shrink the baseline level toward zero.

Exercises

13. For $X^T X = \begin{pmatrix} 2 & 0 \\ 0 & 0.1 \end{pmatrix}$ and $X^T y = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$, compute $\hat{\beta}_{\text{OLS}}$ and $\hat{\beta}_{\text{ridge}}$ at $\lambda = 0.5$. Which coordinate shrinks more, and why?
14. Prove that $X^T X + \lambda I$ is positive definite for every $\lambda > 0$, hence invertible.
15. Show that as $\lambda \rightarrow \infty$, $\hat{\beta}_{\text{ridge}} \rightarrow 0$, and as $\lambda \rightarrow 0^+$ it tends to the OLS solution (when the latter exists).
16. Explain why ridge can return a solution when $p > n$, a regime where OLS has no unique answer.

5 Lasso regression

Ridge shrinks every coefficient but zeroes none. Often we believe instead that **most features are irrelevant** and want the fit to say so by setting their coefficients to **exactly** zero — performing automatic feature selection. A single change, swapping the L_2 penalty for an L_1

penalty, accomplishes this. The change of norm changes the geometry from a round ball to a pointed diamond, and the points are what create the zeros.

5.1 The L1 penalty

Definition 18 (Lasso regression) For $\lambda \geq 0$,

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta \in \mathbb{R}^p} \left(\|y - X\beta\|^2 + \lambda \sum_{j=1}^p |\beta_j| \right).$$

The penalty $\sum_j |\beta_j|$ is the L_1 norm of β . (“Lasso” = **least absolute shrinkage and selection operator**.)

5.2 Why the diamond produces sparsity

In constrained form, lasso minimizes $\|y - X\beta\|^2$ subject to $\sum_j |\beta_j| \leq t$. The constraint region $\sum_j |\beta_j| \leq t$ is a **diamond** (in higher dimensions a cross-polytope): a ball in the L_1 norm, whose **corners sit exactly on the coordinate axes**. As before, the solution is where the expanding elliptical error-contours first touch the constraint region — but now the region has sharp corners, and an expanding ellipse is overwhelmingly likely to strike a **corner** first. At a corner on the β_1 -axis, the coordinate $\beta_2 = 0$ exactly.

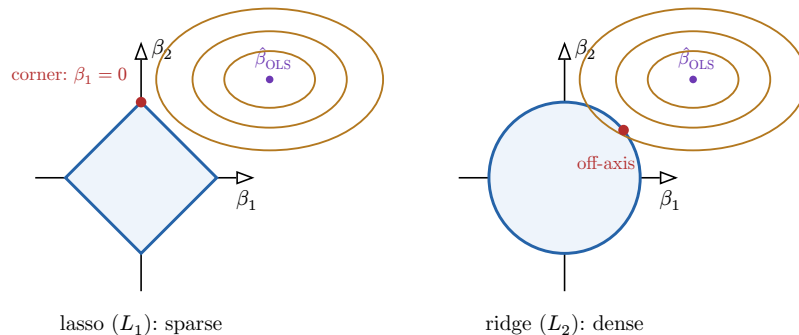


Figure 5: Why L_1 selects and L_2 does not. Left: the lasso diamond has corners on the axes, so the expanding error-ellipses typically first touch at a corner, forcing a coordinate (β_1 here) to exactly zero. Right: the ridge disk is smooth, so the contact point lands off the axes — every coordinate is shrunk but stays non-zero. Geometry, not magic, is what makes lasso a feature selector.

Proposition 19 (Lasso has no closed form) Because $|\beta_j|$ is not differentiable at $\beta_j = 0$, the lasso objective is non-smooth, and there is **no closed-form solution**. The estimate must be computed by an iterative numerical algorithm (coordinate descent, least-angle regression, or proximal-gradient methods).

Remark The non-differentiability is not a defect to be tolerated — it is **exactly the mechanism that creates zeros**. A smooth penalty (like ridge’s) has zero derivative pull at the origin, so it never quite reaches zero; the L_1 penalty has a **kink** at zero with a non-vanishing slope on each side, and that persistent pull is what snaps small coefficients to exactly 0.

Common pitfall Among a group of highly correlated features, lasso tends to keep **one arbitrarily** and zero the rest, and which one it keeps can change with a tiny perturbation of the data. So a zero coefficient means “this feature is not needed **given the others**”, not “this feature is irrelevant”. Read lasso’s selections as a parsimonious model, not as a ranking of importance.

Exercises

17. For the **orthonormal** design $X^T X = I$, the lasso solution has the closed form (soft-thresholding) $\hat{\beta}_{j,\text{lasso}} = \text{sign}(z_j) \max(|z_j| - \lambda/2, 0)$ where $z = X^T y$. Verify it sets to zero exactly the z_j with $|z_j| \leq \lambda/2$.
18. Sketch the L_1 ball $|\beta_1| + |\beta_2| \leq 1$ and the L_2 ball $\beta_1^2 + \beta_2^2 \leq 1$ on the same axes; mark the corners.
19. Explain geometrically why, in three dimensions, the lasso constraint is an octahedron and why its edges and corners produce sparsity even more readily.
20. Why is the soft-thresholding of the first exercise impossible to achieve with the smooth ridge penalty?

6 Comparing L1 and L2 regularization

Ridge and lasso solve the same problem — overfitting and instability — with the same device — a penalty on coefficient size — differing in one choice of norm. That choice has consequences worth stating side by side, so you can pick deliberately.

	Ridge (L_2)	Lasso (L_1)
penalty	$\lambda \sum_j \beta_j^2$	$\lambda \sum_j \beta_j $
constraint	ball (round)	diamond (cornered)
closed form	yes: $(X^T X + \lambda I)^{-1} X^T y$	no — iterative only
coefficients	all shrunk, none zero	many set exactly to zero
selects features?	no	yes
best when	many small effects	few features truly matter

The deciding question is the **sparsity assumption**: do you believe that only a handful of features genuinely drive the response, the rest being noise?

- If **yes** — most coefficients should be zero — prefer **lasso**. Its diamond geometry zeroes the irrelevant features, yielding an interpretable, parsimonious model that does its own feature selection.
- If **no** — many features each contribute a little, and you expect no true zeros — prefer **ridge**. Its smooth shrinkage handles correlated predictors gracefully (shrinking them together rather than arbitrarily picking one), and its closed form is cheap.

Remark You need not choose: the **elastic net** combines both penalties, $\lambda_1 \sum_j |\beta_j| + \lambda_2 \sum_j \beta_j^2$, inheriting lasso’s sparsity while, like ridge, keeping correlated features together. It

is the default in much modern practice — but it is only worth understanding **because** you understand the two pure cases first.

Common pitfall Neither penalty is “more accurate” in the abstract; superiority depends entirely on whether the data match the penalty’s implicit assumption. The honest way to choose between ridge, lasso, and their tuning parameter λ is empirical — cross-validation, the subject of the next chapter — not a prior conviction about which is better.

Exercises

21. For each scenario, state whether ridge or lasso is more appropriate and why: (a) gene-expression data with 20000 genes, of which a few dozen are thought to matter; (b) predicting house price from 8 mildly correlated, all-plausibly-relevant features.
22. Argue geometrically why, with two perfectly correlated features, ridge splits the coefficient evenly between them while lasso loads it all onto one.
23. Write the elastic-net objective and explain which term provides sparsity and which provides stability under correlation.

7 Cross-validation

We have several models on the table — OLS, ridge and lasso at various λ — and need to choose. The temptation is to pick whichever fits the observed data best, but that is precisely the trap. This chapter explains why fitting error lies, and gives the standard honest alternative: estimate each model’s error on data it has never seen.

7.1 Why training error underestimates test error

The estimate $\hat{\beta}$ is chosen to make the error on the **training data** as small as possible. It therefore fits not only the genuine signal but also the particular **noise** in that sample. Measured back on the same data, the model looks better than it is: the **training error** is optimistically biased downward relative to the **test error** — the error on fresh data from the same source. The gap widens as the model grows more flexible: a model with enough parameters can fit the training data perfectly ($R^2 = 1$) yet predict new data terribly. This is **overfitting**, and honest assessment demands data withheld from fitting.

Common pitfall Never report training error as a measure of predictive quality, and never tune λ by minimizing training error. Training error decreases monotonically as the model grows more complex (or as $\lambda \rightarrow 0$), so minimizing it always selects the most complex, most overfit model — the exact opposite of what you want.

7.2 k-fold cross-validation

If we cannot spare a separate test set, we **manufacture** one repeatedly by rotation.

Definition 20 (k-fold cross-validation) Partition the n observations into k disjoint **folds** of roughly equal size. For each $j = 1, \dots, k$:

1. **fit** the model on all folds except fold j (the training set);
2. **evaluate** its mean squared error on the held-out fold j (the validation set), giving MSE_j .

The **cross-validated error** is the average over folds:

$$CV = \frac{1}{k} \sum_{j=1}^k MSE_j.$$

Each observation is used for **validation exactly once** and for **training $k - 1$ times**, so the data are spent efficiently while every error is measured on a point the model did not see.

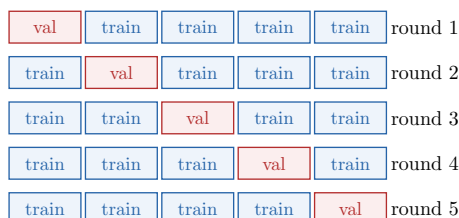


Figure 6: Five-fold cross-validation. The data are split into 5 folds; in each round a different fold is held out for validation (red) while the other four train the model (blue). Every fold serves as validation exactly once. The five validation errors are averaged into a single honest estimate “CV” of out-of-sample performance.

Definition 21 (Leave-one-out CV) **Leave-one-out cross-validation** (LOOCV) is the extreme $k = n$: each fold is a single observation. The model is fit n times, each time predicting the one point left out. It uses the data maximally and is nearly unbiased for the test error, but costs n fits — expensive for large n .

Remark Common practice takes $k = 5$ or $k = 10$: a good compromise between LOOCV’s low bias (much training data each round) and its high cost and high variance. Folds should be assigned at random; for grouped or time-ordered data, the splitting must respect that structure (e.g. never train on the future to predict the past).

7.3 Selecting λ by cross-validation

This gives the principled tuning method promised in Chapters 4–6.

To choose the regularization parameter λ :

1. Fix a grid of candidate values $\lambda_1, \dots, \lambda_m$ (typically spaced logarithmically).
2. For each λ , compute the k -fold cross-validated error $CV(\lambda)$.
3. Select the λ minimizing $CV(\lambda)$ — or, more conservatively, the **largest** λ within one standard error of the minimum (the “one-standard-error rule”), favouring a simpler model.
4. Refit on **all** the data at the chosen λ to get the final coefficients.

The very same procedure chooses among any competing models — polynomial degrees, feature subsets, ridge versus lasso — by comparing their cross-validated errors.

Exercises

24. With $n = 20$ and $k = 5$, how many model fits does k -fold CV require to evaluate a single λ ? How many for LOOCV?
25. Explain why training error cannot be used to choose λ , but cross-validated error can.

26. You evaluate 12 candidate λ values by 10-fold CV. How many total fits is that? Why is a logarithmic grid usually preferable to a linear one?
27. Describe a dataset where ordinary random k -fold CV would give a dishonestly optimistic estimate, and how you would fix the splitting.

8 The bias–variance tradeoff

Cross-validation **measures** out-of-sample error; the **bias–variance decomposition** explains its shape. It splits expected prediction error into two opposing forces — error from a model too rigid to fit the truth, and error from a model too flexible and chasing noise — plus an irreducible floor. Regularization, model complexity, and the U-shaped CV curve are all this one tradeoff seen from different angles.

8.1 The decomposition

Theorem 22 (Bias–variance decomposition) Suppose the truth is $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}(x)$ be a prediction trained on a random sample. The expected squared error at x , averaged over both the noise and the random training sample, decomposes as

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Proof. Abbreviate $\hat{f} = \hat{f}(x)$, $f = f(x)$, and let $|f| = \mathbb{E}[\hat{f}]$. Since ε is independent of $\hat{f}$ with mean 0,

$$\mathbb{E}\left[(y - \hat{f})^2\right] = \mathbb{E}\left[(f + \varepsilon - \hat{f})^2\right] = \mathbb{E}\left[(f - \hat{f})^2\right] + \sigma^2.$$

Now insert and subtract $|f|$:

$$\mathbb{E}\left[(f - \hat{f})^2\right] = \mathbb{E}\left[(f - |f| + |f| - \hat{f})^2\right] = (f - |f|)^2 + \mathbb{E}\left[(|f| - \hat{f})^2\right],$$

because the cross term $2(f - |f|)\mathbb{E}[|f| - \hat{f}] = 0$. The first term is bias^2 , the second is $\text{Var}(\hat{f})$. $\square$

Remark Each term has a plain reading. **Bias** is systematic error: how far the model is wrong **on average** over many training sets — large when the model is too simple to represent f (underfitting). **Variance** is instability: how much the prediction wobbles as the training sample changes — large when the model is too flexible and re-fits the noise (overfitting). The **irreducible** σ^2 is the noise floor no model can beat.

8.2 The tradeoff and its U-shaped curve

The two controllable terms move in **opposite directions** as model flexibility changes:

- **More flexible** (higher polynomial degree, more features, smaller λ): bias falls — the model can bend to the truth — but variance rises — it also bends to the noise.
- **Less flexible** (stronger regularization, larger λ): variance falls but bias rises.

Their sum, the expected test error, is therefore **U-shaped**: high on the left (underfitting, bias-dominated), high on the right (overfitting, variance-dominated), minimized at an intermediate “sweet spot”.

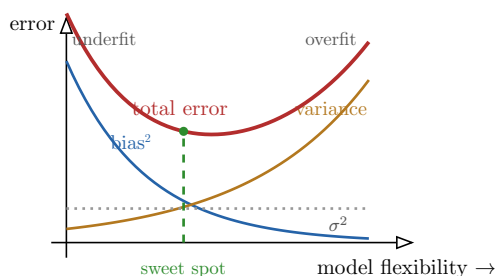


Figure 7: The bias–variance tradeoff. As flexibility increases (rightward, or equivalently λ decreasing), squared bias (blue) falls and variance (orange) rises above the irreducible floor σ^2 . Their sum, the expected test error (red), is U-shaped, minimized at the sweet spot (green). The cross-validation curve $CV(\lambda)$ is an empirical estimate of exactly this red curve — which is why minimizing it finds the right model.

8.3 The numerical demonstration

The OP asks you to **see** this tradeoff, not just state it. The recipe:

To demonstrate the bias–variance tradeoff numerically:

1. Fix a true function f and noise level σ ; the truth is $y = f(x) + \varepsilon$.
2. Choose a knob for flexibility — polynomial degree, or ridge/lasso λ (here larger λ = **less** flexible).
3. Over many simulated training sets, fit the model at each knob setting and record the prediction at a fixed test point.
4. Estimate bias^2 (squared gap between the **average** prediction and f) and variance (spread of predictions) at each setting, and plot both and their sum.

The result reproduces the U-shaped curve: bias^2 decreasing, variance increasing, total minimized in between.

Remark This closes the loop with regularization and estimation theory (MAT31-2). Ridge and lasso deliberately **add bias** (they shrink coefficients away from the unbiased OLS values) in exchange for a **larger drop in variance** — moving you leftward toward the sweet spot. The “variance” here is exactly the estimator variance $\text{Cov}(\hat{\beta}) = \sigma^2(X^T X)^{-1}$ that multicollinearity inflated in Chapter 3: regularization is the cure precisely because it trades a little bias for a large variance reduction.

Common pitfall The “best” model is **not** the one with lowest bias, nor the one with lowest variance, but the one with lowest **sum**. Chasing zero bias (an interpolating, zero-training-error model) lands you at the far right of the curve, with catastrophic variance. The art of modelling is accepting some bias to buy a larger reduction in variance.

Exercises

28. For the constant predictor $\hat{f}(x) = |y|$, argue that bias is large but variance is small. For an interpolating predictor passing through every training point, argue the reverse.
29. Using $\hat{\beta}_{\text{ridge}}$, explain why increasing λ raises bias and lowers variance, and connect this to the shrinkage factors of Chapter 4.
30. Why can the irreducible error σ^2 never be reduced by any model, no matter how flexible?
31. Sketch how the U-curve of total error shifts if the noise level σ increases.

9 The full regression pipeline

Each preceding chapter handled one stage; here they run end to end, the way you would actually analyse a dataset from scratch. Treat this as the checklist that the whole course was building toward.

The from-scratch regression workflow.

1. **Assemble and fit.** Build the design matrix X (intercept column included), standardize the features, and fit OLS via the normal equations $\hat{\beta} = (X^T X)^{-1} X^T y$ — geometrically, project y onto $\text{Col}(X)$.
2. **Diagnose residuals.** Compute $r = y - X\hat{\beta}$; confirm $X^T r = 0$; report R^2 and R^2_{adj} ; inspect the residual plot for curvature (misspecification) or a funnel (heteroscedasticity), and transform or enrich the model if needed.
3. **Check conditioning.** Examine the condition number of $X^T X$ and the variance inflation factors. High values warn that individual coefficients are unstable and regularization is advisable.
4. **Regularize.** Apply ridge $(X^T X + \lambda I)^{-1} X^T y$ for smooth shrinkage of correlated predictors, or lasso under a sparsity assumption for automatic feature selection.
5. **Select λ .** Choose the regularization strength by minimizing the k -fold cross-validated error $\text{CV}(\lambda)$, then refit on all the data.
6. **Interpret.** Read the final coefficients in light of the bias–variance tradeoff — modest, stable, regularized estimates over wild, “unbiased” ones — and report performance using the cross-validated, not the training, error.

Worked example — the pipeline on a small dataset. Suppose we model rent from area, age, and number of rooms ($p = 4$ with intercept), $n = 200$ flats. **Fit:** the normal equations give an OLS line, but **residual analysis** shows a funnel — rents of large flats vary more — so we regress $\log(\text{rent})$ instead and the funnel flattens. **Conditioning:** area and rooms are strongly correlated ($\text{VIF} \approx 8$), so OLS coefficients on those two are unstable and one even comes out negative. **Regularize:** expecting all three features to matter, we choose **ridge**; **cross-validation** over a grid picks λ^* at the minimum of the U-shaped $\text{CV}(\lambda)$ curve. **Refit** at λ^* on all 200 flats: the area and rooms coefficients are now both positive and stable, the model generalizes (its CV error is far below what an overfit OLS would suffer on new flats), and each coefficient is interpretable.

The course in two pictures. Fitting a linear model is **projection**: the fit $\hat{y} = Hy$ is the shadow of y on the column space, the residual the perpendicular gap, and least squares is the shortest perpendicular — everything in Chapters 1–3 (R^2 , orthogonality, multicollinearity as a thin subspace) is this picture. Trusting a model is the **U-curve**: test error falls with flexibility then rises as the model memorizes noise — and Chapters 4–8 (ridge, lasso, cross-validation,

bias–variance) are all about finding its minimum. Master projection and the U-curve and you command the mathematical core of data science: **fitting is geometry, trusting is probability.**